(Blind) Multimodal Bot Detection on X

Hamza Rashid
McGill University
COMP 400

***Abstract*—This research investigates multimodal bot detection on X, in a *blind* prediction setting, where each account is observed exactly once at inference time, and graph information is not available. Our work begins with benchmarking conventional classifiers—a random forest on text, temporal, and topic-distribution features, yielding moderate baseline performance. We then explore various multimodal approaches to combat manipulated features, with an emphasis on DNA-inspired behaviour modeling to encode user activity more dynamically. For this, we embed the user description with TwHIN-BERT and model posting behavior with two DNA formulations, one that captures content across posts, and another capturing inter-post times. We find that CNN embeddings of inter-post time DNA may capture latent temporal patterns that conventional methods miss. We investigate three methods for modality interaction; a GMU (Gated Multimodal Unit), a cross-attention mechanism, and a sparsely gated MoE (mixture-of-experts) using self-attention to acheive modality interaction. We find that the MoE is not only our best method, but is also the smallest of our multimodal models, and does not require any treatment of the class imbalance found in bot detection datasets. Our findings show that a multimodal approach is robust against sophisticated bots, having manipulated features in at least one modality, and that the divide and conquer paradigm helps address the diversity of user communities on X.**

I. Introduction

Social media platforms like X (formerly Twitter) have enabled unprecedented global connectivity. As an open platform is vulnerabe, X is fertile ground for malicious content, often originating from automated accounts—bots—that manipulate discourse, spread misinformation, or artificially amplify content. Detecting and mitigating such bot activities remains a challenging yet essential task for preserving authentic online interactions.

Most current work in bot detection focuses on supervised methods. Yet, labeled data are often non-representative of the evolving behaviors exhibited by sophisticated bots. Consequently, there is a need for detection methods that generalize beyond known bot patterns.

We address this challenge by evaluating a range of bot detection strategies in a *blind* prediction setting, where each account is observed exactly once at inference time. This is acheived through a competition infrastructure where real tweet data guides privately generated bot data in an automated manner.

To detect these bots, we begin with classical machine-learning techniques relying on extensive feature engineering. We employ a random forest incorporating textual features, temporal posting patterns, and inferred topic distributions. While it achieves moderate baseline performance, its reliance on static derived features makes it gullible to manipulation strategies employed by nontrivial bots, such as copying tweets and descriptions from genuine users. This baseline reveals the extensive feature engineering required to keep up with evolving bots under classical approaches.

We thus explore various multimodal architectures to combat manipulated features, with an emphasis on DNA-inspired behaviour modeling to encode user activity more dynamically. Our method combines user description embeddings with posting behavior encoded through two distinct DNA formulations: one capturing content across posts and another encoding inter-post timing patterns. In addition to the known efficacy of content DNA (Ilias et al., 2023), our experiments demonstrate that CNN encodings of temporal-DNA uncover latent behavioral patterns that conventional methods miss.

We explore several techniques for learning cross-modal interactions, including GMUs (Gated Multimodal Unit), cross-attention mechanisms, and a sparsely gated mixture-of-experts (MoE) (Shazeer et al., 2017) using self-attention to acheive modality interaction. Our results highlight the MoE's superior representational capacity and efficiency, notably without requiring strategies to address class imbalance. Our work highlights the robustness of multimodal approaches and dynamic representations against feature manipulation, and illustrates the potential of the MoE architecture to adapt effectively to the diverse user communities found on X.

II. Dataset and Experimental Setup

Our experiments were conducted within the "Bot Or Not" (B.O.N.) competitive infrastructure, designed specifically for assessing bot detection strategies in realistic social media scenarios. This environment featured competing deployments between bot and detector developers, simulating genuine activities on the X platform. The primary goal was to evaluate the effectiveness of detection methodologies under conditions closely resembling live social media dynamics.

A. Data Collection and Characteristics

The dataset comprised real posts gathered from the X API across multiple collection sessions, each lasting the same duration but occurring on different days. For each session, posts were initially collected based on predefined sets of topical keywords relevant to that session. After gathering initial posts, users flagged as obvious bots were removed, and the dataset was expanded by including all additional posts from the remaining users within the session period.

Dataset curation is summarized as follows:

- Text-only Posts: Posts containing images, videos, or memes were excluded.
- Static Metadata: Dynamic metadata such as likes, comments, and follower counts were excluded to maintain consistency and simplicity.
- Anonymized Links and Mentions: Hyperlinks were standardized to "https://t.co/twitter_link", and user mentions were anonymized to "@mention" to prevent easy verification of authenticity.
- Activity Thresholds: Each included user produced between 10 and 100 posts during the session period, ensuring sufficient per-user data.

Datasets were structured into two main JSON formats:

- Posts Dataset: Included text content, timestamp, unique post ID, author ID, and language.
- Users Dataset: Contained user-specific metadata, including user ID, username, display name, profile description, location, tweet count, and a calculated activity-based z-score.

B. Session Structure

Sessions were divided into multiple sequential sub-sessions to mimic the real-time flow of social media feeds, gradually introducing new posts to participants. Bot developers generated content incrementally across these sub-sessions, with the requirement to post between 10 and 100 posts per bot by session completion. Detector deployments subsequently analyzed the fully compiled datasets post-session to identify bots, providing classification along with the bot-confidence score for each account.

C. Technical Setup

Participant submissions (bot and detector code) were managed via public GitHub repositories cloned into Docker images deployed on EC2 instances, ensuring standardized computing resources and fair evaluation conditions. Submissions were required to execute fully within a one-hour timeframe, after which automated shutdown procedures terminated any ongoing processes.

III. Methodology

Our approach involved a systematic progression from classical machine learning methods to multimodal neural-based architectures, overcoming limitations observed at each stage. The absence of graph data and user metadata limited the available strategies, though we explored methods capable of incorporating such features. We aimed to model user behaviour from a variety of perspectives, since nontrival bots manipulate features in at least one aspect (e.g., tweet content, posting times, or interactions such as retweets, likes, and follower and following relationships). The results are shown in table I.

A. *Baseline - Random Forest*

For our baseline study, we employ a random forest for its flexibility with data types, allowing rapid feature exploration, and proven efficacy in bot detection, most notably in Botometer (Davis et al., 2016).

The average ratio of humans to bots across the sessions is roughly $14:1$, making the datasets moderately imbalanced. Our random forest used class weights inversely proportional to class frequencies to improve bot recall. It consisted of 100 trees, and was trained and evaluated using stratified 80-20 train-test splits.

The sub-approaches below correspond to progressively enriched feature sets, assessing how well feature-based models generalize across batches of social media activity.

1) ***Session 11*** *– Embeddings + LDA:* For session 11, we extracted tweet embeddings for each user with all-MiniLM-L6-v2 (Reimers and Gurevych, 2021) to capture textual behaviour. Further, we used LDA (Blei et al., 2003) topic modeling under the assumption that bots target a few topics disproportionately, resulting in vectors with unusually large values in a few entries, while those of genuine users may also be sparse, but less concetrated. Aggregating posts into a single document simplified the problem but at the cost of granularity, which we later addressed. We performed minor text preprocessing by converting URLs, mentions, and hashtags into special tokens (`<URLURL>`, `<UsernameMention>`, `<HashtagMention>`) in order to remove noise from LDA, which relies on term frequencies. LDA was performed with stopword exclusion performed internally, leaving the contextual tweet embeddings unaffected.

Result: The model exbihibted low confidence and marked all samples as negatives. This likely owes to LDA's reliance on predefined topics from training data, which vary across sessions and thus produce sparse vectors on unseen data.

Limitations:

- Did not use individual tweets as independent samples, thereby lacking granulrity.
- Static topic modeling limited generalization to new sessions.

2) ***Session 12*** *– Mean Embedding + BERTopic + Temporal Statistics:* In this session, we shifted to using per-tweet embeddings and adopted BERTopic (Grootendorst, 2022) for more dynamic topic modeling. Additional temporal statistics were introduced to capture behavioral rhythms. As an extra measure for addressing the low confidence scores, we performed min-max scaling of the confidences after inference.

Implementation Summary:

- Tweet-Level Processing: Individual tweets were embedded and clustered into topics using BERTopic with HDBSCAN (McInnes et al., 2017).
- Aggregated features: Mean of tweet embeddings and normalized topic distribution.
- (Naive) Temporal features: Mean and standard deviation of raw timestamps.
- Static features: Post count.

Result: The model acheieved moderate results in this session, improving in both recall and precision. This result likely owes to (1) scaled confidence scores, (2) time based

features providing obvious signals and (3) batch-agnostic topic modelling providing informative features to the classifer. The time-based features likely provided the most utility, since many bots at this point did not modify their posting times.

Limitations:

- Topic feature vectors were padded to a fixed length, potentially introducing sparsity and noise.
- Temporal features were still limited to coarse statistics.

3) ***Session 13*** *– Enhanced Temporal Behavior Modeling:* Session 13's approach can be summarized as follows:

- Building on Session 12, we added:
- Inter-post Interval Statistics: Mean and standard deviation of time gaps between consecutive posts.
- Bucketed Duplicate Counts: Timestamps were partitioned into 10 intra- and inter-chronologically-ordered buckets, and duplicate timestamps were computed per bucket.
- Features Used: Mean-pooled tweet embeddings, topic distributions, post frequency, inter-post intervals, and bucketed duplicates.

Result: This was the strongest-performing baseline. The bucketed duplicates introduced a dynamic temporal feature, and using consecutive-post time gaps to compute the temporal statisics was more appropriate than using raw timestamps. We suspect that the bots may have used a randomized post scheduler, resulting in a larger mean and variance in the time-gaps. An area of improvement would be to explicitly model inter-feature dependencies (e.g., topic-time correlations).

4) ***Session 14****– Topic Statistics + Temporal Statistics + Intra-Topic Similarity:* This final Random Forest setup expanded feature engineering to include statistical, temporal, and semantic regularities observed in bot behavior.

Implementation Summary:

- Hashtags were partially preserved (`#arena` → `<HashtagMention:arena>`), preserving topic cues often used by bots.
- Tweet embeddings were explicitly normalized to prevent variance from dominating downstream signals.
- Topic Modeling: BERTopic + HDBSCAN, followed by:
  - Topic Histogram (topic mention counts across topics)
  - Topic Statistics: Mean, variance, and standard deviation across topic indices.
- Temporal Signals:
  - Mean, standard deviation, and variance of time between posts.
  - Bucketed duplicate timestamps (tolerance-aware) to flag spam bursts.
- Intra-topic Semantic Consistency:
  - Mean, median, and standard deviation of pairwise cosine similarities across tweets within each user-topic group.

Result: In session 14, the bots became more difficult to distinguish. That is, this model outperformed its predecessor under the same training conditions when applied to the previous session, but performed worse on the latest. Despite the drop in performance, the model acheived nontrivial results, second only to the previous session. A core limitation was the increased feature dimensionality, introducing noise to the model and potentially stressing interpretability.

B. Summary and Reflection on Random Forest Baseline

Our progression through four sessions using Random Forest classifiers illustrates the evolving challenge of distinguishing bots from humans on X through feature engineering alone.

What Went Right:

- Transitioning from static LDA to BERTopic enabled models to better adapt to new sessions.
- Behavioral Signals: Features such as inter-post intervals and intra-topic semantic consistency offered important signals for detecting scripted or bursty behavior patterns.

What Went Wrong:

- The varying topics across sessions might have created inconsistencies in how distributional topic-features (e.g., histogram) were interpreted by the classifier.
- Scalability Concerns: Computing transformer embeddings, pairwise similarities, multiple topic statistics, and padded histograms scales poorly for large user datasets.

While the Random Forest baseline allowed us to test many ideas rapidly, increasing complexity of feature engineering was needed to keep up with evolving bots. They often copied tweets and user descriptions from genuine users, thereby reducing informative signals in our features. Further, while we were able to encode user behaviour from various perspectives, our model relied on many static features (e.g., mean and standard deviation of time between posts), which tend to be the most vulnerable to feature manipulation.

The observed manipulation and feature engineering upkeep motivated the shift to multimodal methods that can better adapt to evolving behaviours on X.

C. ***Multimodal Architectures***

Recognizing the constraints of feature engineering, we implement multimodal neural architectures inspired by recent literature *"Multimodal Detection of Bots on X (Twitter) using Transformers"* (Ilias et al., 2023), and *"BotMoE: Twitter Bot Detection with Community-Aware Mixtures of Modal-Specific Experts"* (Liu et al., 2023). We utilize user description and tweet embeddings alongside digital DNA representations (content and temporal).

We explore the following designs for capturing cross-modal interactions:

- Gated Multimodal Unit (GMU): Uses a gating mechanism to learn modality interactions.
- Cross-Attention: Uses multihead attention to learn modality interactions.
- Mixture-of-Experts (MoE): Dedicates a MoE model for each modality, subsets of the MoE specialize in different user types, and self-attention is used to acheive cross-modal interactions.

These architectures not only provide greater modeling capacity, but also offer more robustness against feature manipulation, as they do not rely on manually crafted statistics. Instead, they operate directly over sequences of observed user behavior and learn to extract structure.

*1) **DNA Embedding Extraction**:* To model user behavior beyond text embeddings, we construct symbolic sequences known as digital DNA, across the user's timeline chronologically, representing content, timing, and emoji usage across tweets. These sequences are then converted into images and embedded via a CNN encoder. In all cases, the resulting character sequence is transformed into an image as follows:

1) Each symbol is mapped to an 8-bit grayscale intensity value (e.g., `N` $\mapsto$ 0, `U` $\mapsto$ 64, ..., `X` $\mapsto$ 255).
2) The sequence of grayscale values is reshaped into a square matrix: if the sequence has length L, the smallest $n \in \mathbb{N}$ such that $n^2 \geq L$ is selected. The sequence is zero-padded to length n^2 and reshaped to $(n \times n)$.
3) This grayscale image is resized depending on the CNN encoder, using nearest-neighbor interpolation.
4) The image is then converted to RGB format by replicating the grayscale channel, and the resulting tensor has shape: $3 \times$ `image_width` $\times$ `image_height`.
5) The tensor is normalized using ImageNet statistics and passed through a pretrained CNN encoder.
6) The embedding is obtained by either (1) projecting a mean-pooling of the last hidden layer, or (2) directly extracting the last hidden layer if it is 1D.

Content DNA. Each tweet is mapped to a symbol based on the type of entities it contains:

- `N` — No entities (plain text).
- `U` — Contains a URL.
- `H` — Contains one or more hashtags.
- `M` — Contains one or more mentions.
- `X` — Contains a combination of multiple entity types.

Time DNA. Assigns symbols based on the time elapsed between consecutive tweets:

- `n` — User's first post (no delta).
- `o` — $<$ 1 second since the user's previous post.
- `s` — $\geq$ 1 second and $<$ 1 minute.
- `m` — $\geq$ 1 minute and $<$ 1 hour.
- `h` — $\geq$ 1 hour and $<$ 6 hours.
- `u` — $\geq$ 6 hours and $<$ 12 hours.
- `x` — $\geq$ 12 hours.

Emoji DNA. We categorize emojis in tweets using a precompiled lookup table. Each tweet is assigned a category based on its emojis:

- `S` — Smileys & Emotion.
- `P` — People & Body.
- `A` — Animals & Nature.
- `F` — Food & Drink.
- `T` — Travel & Places.
- `R` — Activities.
- `O` — Objects.
- `Y` — Symbols.
- `L` — Flags.
- `N` — No emoji present.
- `X` — Multiple emoji categories.

In all cases, these DNA images serve as behavioral *fingerprints*, allowing CNNs to learn latent signals over them.

*2) **Gated Multimodal Unit (GMU)**:* The GMU allows the network to dynamically control how much each modality contributes to the final hidden representation. The output is used (with possibly further transformation) downstream for classification.

In the bimodal setting, where we consider two input modalities $\mathbf{x}_v$ and $\mathbf{x}_t$ (e.g., DNA and text), the equations governing the GMU are as follows:

$$\mathbf{h}_v = \tanh(\mathbf{W}_v \cdot \mathbf{x}_v) \tag{1}$$

$$\mathbf{h}_t = \tanh(\mathbf{W}_t \cdot \mathbf{x}_t) \tag{2}$$

$$\mathbf{z} = \sigma(\mathbf{W}_z \cdot [\mathbf{x}_v, \mathbf{x}_t]) \tag{3}$$

$$\mathbf{h} = \mathbf{z} * \mathbf{h}_v + (1 - \mathbf{z}) * \mathbf{h}_t \tag{4}$$

$$\mathbf{\Theta} = \{\mathbf{W}_v, \mathbf{W}_t, \mathbf{W}_z\} \tag{5}$$

Here, $\mathbf{h}_v$ and $\mathbf{h}_t$ are hidden activations learned on modalities $\mathbf{x}_v$ and $\mathbf{x}_t$, $[\cdot, \cdot]$ denotes concatenation, and $\mathbf{\Theta}$ denotes learnable parameters. Cross-modal interactions are learned in the parameters $\mathbf{W}_z$, the weighting mechanism $\mathbf{z}$ then determines how much each modality contributes to the fused representation $\mathbf{h}$. In practice, implementations tend to depart[1] from some of the original equations, including ours[2].

Resembling the work of Ilias et al. (2023), our intial efforts with the GMU used the following modalities:

- Text: User description embedding from TwHIN-BERT (Zhang et al., 2022).
- DNA: Content-DNA embedding from MobileNetV2 (Sandler et al., 2018).

The first iteration consisted of a two-layer classification head applied to the GMU's output. This model marked all samples as negatives, and several adjustments were needed. Subsequent attempts introduced refinements such as: two-layered modality-specific projections, layer normalization, global residual connections, and modality-specific dropout.

*a) **Incorporating Time-DNA**:* While refining our GMUs, we incorporated time-DNA embeddings as a third sub-modality. This addition improved performance, supporting our intuition that a dynamic temporal feature can help capture richer distinctions between bots and humans.

While our GMU iterations demonstrated nontrivial improvements, development required numerous adjustments to stabilize training and optimize performance. This raised concerns about generalizability; can such a carefully balanced architecture handle different modalities or scale to more modalities?

[1]Demonstration https://github.com/johnarevalo/gmu-mmimdb/blob/master/model.py by Arevalo et al. (2017) omits modality projections and applies nonlinearity directly to the inputs.

[2]Concatenates the hidden activations.

We thus employed cross-attention for learning modality interactions, expecting that our model could acheive nontrivial performance with fewer architectural tweaks.

*3) **Cross-Attention**:* We first learn an intermediate representation of the DNA embeddings (content and time) to later apply cross-attention with the user description. We found that the GMU learned fused-DNA representations effectively, despite its intended multimodal use.

Our cross-attention method (illustrated in Figure 1) fuses text (user description) and DNA in the following way:

- Apply a simple GMU to the DNA embeddings to create an intermediate representation of fused-DNA.
- Apply cross-attention layers to the description and fused-DNA in both directions.

To perform cross-modal attention, we employ two multihead-attention layers, computing scaled dot-product attention (Vaswani et al., 2017)

$$\text{Attention}(Q, K, V) = \text{softmax}(\frac{QK^T}{\sqrt{d_k}})V \tag{6}$$

in one direction where the description modality is the query (Q) and fused-DNA provides the key (K) and value (V), and another direction in which the modalities swap roles. Here, d_k denotes the dimensionality of the query and key. A mean pooling is then applied to attention outputs to form a final fused representation.

With the fused representation, we began with a two-layer classification head and iteratively improved our architecture. This required less refinements than our GMU approach.

*a) **Sessions 16-17** - Description, Content, Time:* For sessions 16 and 17, we employed the cross-attention approach, and trained the model with weighted cross-entropy loss to deal with the class imbalance. The model's design follows a structured flow employing conventional methods such as residual connections and layer normalization:

- Pre-Layer normalization (Xiong et al., 2020) and residual connections are applied at each attention stage.
- The cross-attention outputs are concatenated into a sequence and fed through a single position-wise FFN for better regularization.
- Pre-layer normalization is applied to the tokens and a residual connection is added after the FFN output.
- A mean pooling is applied to the FFN outputs, and a two-layer classification head generates the final logits for bot detection.

The model underperformed relative to some of our random forest attempts. However, the bots had improved significantly by session 16, and a drop in performance was expected given the model's complexity and the limited fine-tuning conducted at this stage. However, the model showed strong cross-lingual transfer, acheiving the highest recall in French (session 17) despite training only on English data up to this point, demonstrating the importance of a multimodal approach.

*b) **Sessions 18-19** - Description, Content, Time, Emoji:* Noticing that LLM-powered bots exhibited obvious tells in their emoji usage across posts (e.g., repeatedly using a single category), the model's GMU was extended to fuse three DNA embeddings for sessions 18 and 19: content, time, and emoji. The model was trained with focal loss (Lin et al., 2017), which was more effective than weighted cross-entropy for dealing with the class imbalance.

Bot recall did not improve, but since we had more time to fine-tune the model, we achieved a higher precision and f1 score than our previous attempt. The sparse and high-dimensional nature of the emoji embeddings may have introduced noise to the model, which would explain the increased sensitivity to hyperparameter configurations. We therefore removed the emoji-DNA from subsequent approaches.

This experiment revealed the challenges of integrating sparse representations like emojis. Future refinements may involve curating a more appropriate categorization system for emojis.

c) Conclusion: The cross-attention approach showed promising results, with the highest recall in session 17, but due to the tuning requirements and use of emoji-DNA, it underperformed in the other sessions.

Although a cross-attention approach can be effective, it is computationally expensive to scale beyond two modalities. For example, three modalities would require six multihead-attention layers to consider each bimodal interaction in both directions.

A natural alternative is to employ a single self-attention layer where each token corresponds to a modality. A key challenge in this approach is learning modality representations to provide to the self-attention layer. We find that MoEs learn these representations effectively, and in particular, might be better at modeling the diverse user behaviours found on X.

*4) **Mixture-of-Experts (MoE)**:* To address the challenges of (1) model scalability and (2) diverse user behavior on X, we employ the sparsely gated Mixture-of-Experts (MoE) (Shazeer et al., 2017) strategy used by BotMoE (Liu et al., 2023). This approach leverages specialized experts conditioned on (learned) user sub-communities, enabling community-aware representation learning.

a) Expert Assignment and Gating: For each modality, user embeddings are fed into a **gating network**:

$$G(\mathbf{x}) = \text{softmax}(\text{KeepTopK}(W_g\mathbf{x}, k)) \tag{7}$$

where W_g are learnable weights, $\mathbf{x}$ is the input embedding, and `KeepTopK` retains the top k gating scores for sparse expert activation.

b) Expert Aggregation: The final representation for modality $m \in \{t = \text{text}, d = \text{DNA}\}$ is a weighted sum of expert outputs:

$$\mathbf{z}_i^{(m)} = \sum_{j=1}^{n} G(\mathbf{x}_i^{(m)})_j E_j(\mathbf{x}_i^{(m)}) \tag{8}$$

where $E_j(\cdot)$ denotes the j^{th} expert (a two-layer MLP), and $G(\mathbf{x}_i^{(m)})_j$ is the gating score for expert j.

c) Feature Fusion and Consistency: After expert aggregation, representations across modalities are fused using a multi-head **self-attention** mechanism:

$$\{\tilde{\mathbf{z}}^{(t)}, \tilde{\mathbf{z}}^{(d)}\} = \text{SelfAttn}(\{\mathbf{z}^{(t)}, \mathbf{z}^{(d)}\}) \tag{9}$$

Additionally, we evaluate the consistency between the modality representations.

$$\mathbf{z}^{(con)} = \text{flatten}(\text{Conv2D}(\text{sample}(\mathbf{M}))) \tag{10}$$

This is done by applying a 2D convolution[3] on a downsampling (via fixed pooling) of the attention matrix $\mathbf{M}$.

d) Classification Head: The fused representations and consistency vector are concatenated and passed through a final classifier:

$$y_i = W_o \cdot [\tilde{\mathbf{z}}^{(t)}, \tilde{\mathbf{z}}^{(d)}, \mathbf{z}^{(con)}] + b_o \tag{11}$$

where W_o and b_o are learnable parameters.

e) Loss Function: We optimize a cross-entropy loss with two regularization terms:

$$\mathcal{L} = -\sum_{i \in \mathcal{Y}} t_i \log y_i + \lambda_1 \sum_{\theta} \|\theta\|_2^2 + \lambda_2 \sum_{i \in \mathcal{Y}} \sum_{m \in \{t,d\}} BL(\mathbf{x}_i^{(m)}) \tag{12}$$

where θ denotes model parameters (used for L2-regularization) and $BL(\cdot)$ is the **balancing loss** from Shazeer et al. (2017):

$$BL(\mathbf{x}) = w_{imp} \cdot CV(G(\mathbf{x}))^2 + w_{ld} \cdot CV(P(\mathbf{x}, i))^2 \tag{13}$$

Here, CV is the coefficient of variation, encouraging balanced expert usage.

*f) **Sessions 20-21** - Text, DNA:* For sessions 20 and 21, we employed the outlined MoE approach with the following modalities:

- Text: User description + Five tweets sampled evenly across their timeline (partitioning their posts into equal chunks), both encoded with TwHIN-BERT.
- DNA: Content + Time, encoded with MobileViT-small (Mehta and Rastegari, 2021).

The architecture (broadly illustrated in Figure 2) can be summarized as follows:

- Each modality is given a 2-expert MoE, and only the top expert is chosen (i.e., $k = 1$ in (7)).
- The resulting outputs interact via multi-head self-attention.
- The concatenated representation outlined in (11) is passed through a one-layer classification head.

Strengths:

- *Efficiency*: It is the smallest of our multimodal models, requiring less time to train and perform inference.
- *Community-aware specialization*: Each expert learns patterns within specific user sub-communities, improving generalization across diverse bot behaviors, and potentially simplifying the learning process due to reduced per-expert user diversity.

[3]Our implementation (sessions 20-21), similar to BotMoE https://github.com/lyh6560new/BotMoE, uses the fixed-pooling output directly.

Challenges:

- *Hyperparameter tuning:* Despite being a smaller model, the sparsely gated MoE architecture has several hyperparameters (e.g., expert hidden dimension, distribution for the gates) requiring careful tuning.

The MoE was our strongest model, acheiving perfect precision and the highest recall in session 20 (English), and the highest recall in all of the French sessions. We reason that the model did not require treatment of class imbalance because the reduced per-expert user-diversity made it easier for experts to learn informative representations.

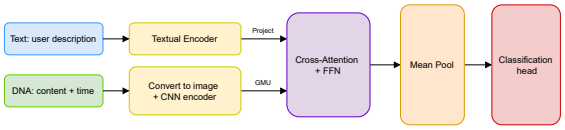

Fig. 1: Text-DNA Cross-Attention bot detection framework.

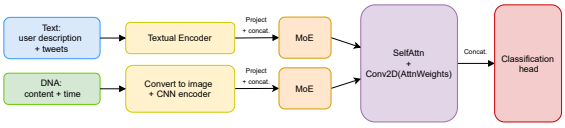

Fig. 2: Text-DNA MoE bot detection framework.

IV. Discussion

In this section we interpret the results in Table I and discuss how they illuminate the challenges of detecting increasingly sophisticated bots on X. We frame the discussion around three axes that emerged as decisive throughout the competition lifecycle: (i) the tension between bots' **manipulated static features** and detectors' **dynamic behaviour-centric features**, (ii) the **diversity of user communities**, and (iii) the **scalability** of detection architectures.

A. Key empirical trends

The baseline Random Forests (RF) established in Sessions 11–14 offer an instructive point of comparison: although overall accuracies exceeded 95% across the four sessions, bot *recall* was volatile, ranging from 0% (Session 11) to 90.5% (Session 13). This disparity highlights a pitfall of static, heavily engineered features in highly imbalanced settings, with high accuracy coexisting with near-zero sensitivity to the minority class.

Transitioning to the *Cross-Attention* (CA) model in Sessions 16–19 reverses this failure mode: recall stabilises between 52% and 72%, but precision remains modest (14–62%), despite the addition of noisy features (emoji-DNA). The shift underscores the benefit of multmodal representation learning, but also reveals the difficulty of learning rich cross-modal interactions with evolving bots and imbalanced labels.

The MoE architecture deployed in Sessions 20–21 marks a decisive improvement. Perfect precision (100%) and a recall of 67.7% in Session 20 demonstrate that community-aware

specialization can reconcile the precision-recall trade-off without explicit re-balancing of the loss. The performance uplift is achieved with fewer parameters than the cross-attention approach, showing that a divide-and-conquer architecture can capture heterogeneous behaviours more effectively than large monolithic models.

B. *Manipulated* vs. *dynamic features*

Early sessions revealed that bots actively engaged in feature manipulation strategies, such as copying profile descriptions and tweet content from geniune users. Consequently, feature sets anchored to surface-level statistics (mean embeddings, global topic histograms, etc.) suffered from feature dilution. Digital DNA representations address this weakness by implicitly encoding temporal dynamics that are harder to spoof (but still vulnerable to manipulation, hence the need for a multimodal approach). CNN embeddings of DNA images capture higher-order patterns (e.g., periodic behaviour) beyond the reach of coarse summary statistics. The progressive gain from Session 12 to 14 (RF models) already hinted at the importance of dynamic signals; the multimodal results confirm that learning these patterns directly, rather than engineering them, yields more robust detectors.

C. *User diversity and community-aware modeling*

Bots on X do not form a homogeneous cluster; they range from single-issue spam accounts to multilingual LLM-powered impersonators. Treating all users the same can therefore lead to weak detectors. The MoE's sparsely gated experts act as implicit community detectors, allowing the model to attend to behaviourally coherent sub-populations (e.g., language communities, topical niches). The cross-lingual recall gains observed in French (Session 21) and the balanced expert utilisation encouraged by the balancing loss are practical evidence of this effect.

D. *Scalability and computational trade-offs*

Multimodal learning can be computationally expensive. While the CA model removed feature-engineering overhead, its computational costs grow quickly with additional modalities. In contrast, the MoE approach keeps the core attention footprint small and delegates specialisation to lightweight MLP experts. This efficiency is crucial for practical applications.

V. Conclusion

This study set out to detect social bots on X in a *blind* prediction setting, where each account is observed exactly once at inference time and no graph information is available. We began with classical Random Forest baselines that relied on hand-engineered, largely static features. While these models occasionally achieved impressive accuracy, their brittleness against feature manipulation became clear when bots copied surface cues from genuine users, such as tweets, and when LLM-powered bots underwent fine-tuning to generate text sounding more human-like.

Session	Model	Total Users	Acc.	Prec.	Rec.	F1
11	RF	420	0.9929	0.0000	0.0000	0.0000
12	RF	393	0.9669	0.7333	0.5500	0.6286
13	RF	394	0.9949	1.0000	0.9048	0.9500
14	RF	402	0.9577	0.7143	0.6897	0.7018
15	N/A	–	–	–	–	–
16	CA	438	0.8151	0.1429	0.5714	0.2286
17	CA	306	0.8105	0.1970	0.7222	0.3095
18	CA	455	0.9275	0.5714	0.5263	0.5479
19	CA	288	0.9132	0.6176	0.6364	0.6269
20	MoE	448	0.9777	1.0000	0.6774	0.8077
21	MoE	283	0.9682	0.8800	0.7857	0.8302

TABLE I: Performance of our bot detectors across sessions. Precision, recall, and F1 are computed for the bot class.

Our multimodal exploration revealed notable strategies. First, behaviour-centric representations such as content- and time-DNA capture longitudinal patterns—burstiness, scheduling regularities, and entity-mixing—that are tedious for bots to imitate consistently, thereby hardening the detector against superficial mimicry. Second, the sparsely gated, community-aware Mixture-of-Experts architecture narrows the usual precision–recall trade-off: by routing behaviourally similar users to the same specialist, each expert faces reduced intra-class variability and can learn sharper decision boundaries, yielding higher recall without sacrificing the perfect precision we observed, all while keeping the model lightweight. Across the final two sessions the MoE attained perfect precision and the highest cross-lingual recall, despite being the smallest of the neural models we evaluated.

Taken together, these findings suggest two design strategies for blind bot detection: (i) Use dynamic, longitudinal signals. (ii) Allocate model capacity adaptively—specialised experts for heterogeneous user sub-populations.

Overall, divide-and-conquer multimodal detectors that embrace behavioural dynamics offer a promising defence against evolving social bots. The MoE framework demonstrates that such models can be simultaneously effective and computationally efficient, which are desirable properties for real-world applications in bot detection.

References

John Edison Arevalo, Thamar Solorio, Manuel Montes-y G'omez, and Fabio A. Gonz'alez. Gated multimodal units for information fusion. *CoRR*, abs/1702.01992, 2017. URL https://arxiv.org/abs/1702.01992.

David M. Blei, Andrew Y. Ng, and Michael I. Jordan. Latent dirichlet allocation. *Journal of Machine Learning Research*, 3:993–1022, 2003.

Clayton A. Davis, Onur Varol, Emilio Ferrara, Alessandro Flammini, and Filippo Menczer. Botornot: A system to evaluate social bots. In *Proceedings of the 25th International Conference Companion on World Wide Web*, pages 273–274. ACM, 2016.

Maarten Grootendorst. Bertopic: Neural topic modeling with a class-based tf-idf procedure. *arXiv preprint*

arXiv:2203.05794, 2022. URL https://arxiv.org/abs/2203.05794.

Loukas Ilias, Ioannis Michail Kazelidis, and Dimitris Th. Askounis. Multimodal detection of bots on x (twitter) using transformers. *arXiv preprint arXiv:2308.14484*, 2023. URL https://arxiv.org/abs/2308.14484.

Tsung-Yi Lin, Priya Goyal, Ross Girshick, Kaiming He, and Piotr Dollár. Focal loss for dense object detection. In *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*, pages 2980–2988. IEEE, 2017. doi: 10.1109/ICCV.2017.324. URL https://arxiv.org/abs/1708.02002.

Yuhan Liu, Zhaoxuan Tan, Heng Wang, Shangbin Feng, Qinghua Zheng, and Minnan Luo. Botmoe: Twitter bot detection with community-aware mixtures of modal-specific experts. In *Proceedings of the 46th International ACM SIGIR Conference on Research and Development in Information Retrieval*, SIGIR '23, pages 485–495, Taipei, Taiwan, July 2023. ACM. doi: 10.1145/3539618.3591646.

Leland McInnes, John Healy, and Steve Astels. hdbscan: Hierarchical density based clustering. *The Journal of Open Source Software*, 2(11):205, 2017. doi: 10.21105/joss.00205.

Sachin Mehta and Mohammad Rastegari. Mobilevit: Lightweight, general-purpose, and mobile-friendly vision transformer. *arXiv preprint arXiv:2110.02178*, 2021. URL https://arxiv.org/abs/2110.02178.

Nils Reimers and Iryna Gurevych. Making monolingual sentence embeddings multilingual using knowledge distillation. https://www.sbert.net, 2021. Accessed: 2025-04-25.

Mark Sandler, Andrew Howard, Menglong Zhu, Andrey Zhmoginov, and Liang-Chieh Chen. Mobilenetv2: Inverted residuals and linear bottlenecks. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 4510–4520, 2018. doi: 10.1109/CVPR.2018.00474. URL https://arxiv.org/abs/1801.04381.

Noam Shazeer, Azalia Mirhoseini, Krzysztof Maziarz, Andy Davis, Quoc V. Le, Geoffrey E. Hinton, and Jeff Dean. Outrageously large neural networks: The sparsely-gated mixture-of-experts layer. *CoRR*, abs/1701.06538, 2017. URL http://arxiv.org/abs/1701.06538.

Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems*, pages 5998–6008. Curran Associates, Inc., 2017. URL https://papers.nips.cc/paper_files/paper/2017/file/3f5ee243547dee91fbd053c1c4a845aa-Paper.pdf.

Ruibin Xiong, Yunchang Yang, Di He, Kai Zheng, Shuxin Zheng, Chen Xing, Huishuai Zhang, Yanyan Lan, Liwei Wang, and Tie-Yan Liu. On layer normalization in the transformer architecture. *arXiv preprint arXiv:2002.04745*, 2020. URL https://arxiv.org/abs/2002.04745.

Haoyi Zhang, Yuhan Liu, Qinghua Zheng, and Minnan Luo. Twhin-bert: A robust twitter user representation model for social bot detection. In *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 2879–2893. Association for Computational Linguistics, 2022. doi: 10.18653/v1/2022.emnlp-main.210. URL https://aclanthology.org/2022.emnlp-main.210.